Native FreeToken Serving on AMD Strix Halo: A ROCm/HIP Port and Controlled Unified-Memory Evaluation

FreeToken AMD contributors

Anonymized review copy; correspondence through the project repository.

Technical white paper, release candidate v0.1.0-rc1, 30 August 2026

ABSTRACT

Large mixture-of-experts (MoE) models make capable local inference possible, but most edge-serving systems are designed and evaluated on NVIDIA discrete GPUs. We present a native ROCm/HIP port of FreeToken for AMD Strix Halo, a unified-memory APU platform represented by the Ryzen AI Max+ 395 with Radeon 8060S graphics (`gfx1151`). The port retains FreeToken's CUDA behavior while adding HIP extension builds, ROCm-safe architecture detection, portable Triton paths, and native model-loading and serving validation. It executes without a CUDA compatibility layer, Vulkan substitute, or CPU-only fallback. We evaluate the port on a GMKtek EVO-X2, a Strix Halo system with 64 GiB installed LPDDR5 memory and a 4 GiB firmware GPU reservation. Linux exposes 59.46 GiB host memory and ROCm exposes a 56.0 GiB coarse-grained GPU pool. We use Qwen3.6-35B-A3B and Gemma 4 26B A4B controls. The port serves Qwen's NVIDIA NVFP4 checkpoint through an OpenAI-compatible streaming API and reproduces a deterministic AIME canary with the reference router at 27.88 mean client-visible decode tokens/s. A faster NVFP4 Triton-router path was rejected because it changed deterministic model output. For a matched raw-prompt Q4_K_M Qwen control, both FreeToken and llama.cpp used the same 54-token prompt and produced the correct mathematical result; FreeToken reached 50.63 tokens/s after enabling a quality-checked native HIP router, compared with 50.29 tokens/s for the ROCm 10 llama.cpp control. A Gemma 4 Q4 text control reached 57.05 tokens/s and returned the expected deterministic answer. These results establish functionality and a bounded same-file Q4 control, not a strict reproduction of FreeToken's published 39.3 tokens/s RTX 4060 result. The upstream prompt corpus, cache state, stop policy, exact revision, and configuration remain incomplete. Profiling instead identifies dense mixed-FP8 decode as the dominant NVFP4 Qwen kernel consumer and shows that a worst-case unified-memory expert-cache fill is materially smaller than end-to-end token time. We release the porting boundary, validation contract, and rejected-candidate evidence to make AMD edge-serving claims reproducible and falsifiable.

1. Introduction

Open-weight MoE models are increasingly capable, yet practical local serving remains concentrated on systems with CUDA-capable discrete GPUs. FreeToken demonstrated that MoE-aware placement, caching, and execution policies can turn consumer hardware into a viable local serving platform [1]. Its design assumes the practical realities of edge inference: model state frequently exceeds device memory, execution alternates between prefill and decode, and agentic workloads repeatedly edit and extend context.

AMD Strix Halo changes an important part of that deployment model. Its Radeon 8060S GPU and CPU share a large LPDDR5X memory pool rather than communicating through a discrete-GPU PCIe path. This makes large local models feasible on an APU, but it does not make a CUDA-oriented serving runtime automatically portable or performant. The runtime must compile native extensions with HIP, avoid treating HIP's `torch.cuda` compatibility namespace as evidence of NVIDIA hardware, preserve model semantics across alternate kernels, and measure CPU-GPU contention rather than assuming PCIe transfer is the principal cost.

This work asks a narrower question than the original FreeToken paper: can FreeToken's serving stack be ported to a Strix Halo `gfx1151` system as a native ROCm/HIP runtime, and what do controlled model-serving experiments show after the port? We make four contributions:

1. We implement a narrowly gated ROCm/HIP port that preserves CUDA behavior and compiles native extensions for `gfx1151`.

2. We define a validation contract that separates native execution, API correctness, deterministic output, matched controls, and paper-parity claims.

3. We report controlled Qwen and Gemma results with explicit prompt, model-format, and metric boundaries.

4. We use native ROCm profiling and cache-copy experiments to identify the current optimization frontier and report rejected candidates instead of presenting microbenchmark wins as system improvements.

2. Background and Porting Challenges

FreeToken targets local MoE serving by jointly managing model layout, expert residency, CPU-GPU execution, and cache state [1]. Its published Qwen3.6-35B-A3B result reports 39.3 decode tokens/s on an 8 GiB RTX 4060 laptop. That result uses the official NVIDIA NVFP4 release, and the paper reports per-request mean decode throughput and mean time to first token (TTFT) across agentic workloads [1].

The target here differs in both hardware and software. The evaluated system is an AMD Ryzen AI Max+ 395 system with Radeon 8060S graphics, `gfx1151`, 64 GiB installed LPDDR5 memory, and a firmware-reserved integrated-GPU allocation. Its ROCm 10 runtime and HIP compiler enable native execution, but FreeToken contains CUDA-specific extension, JIT, architecture-detection, and Triton assumptions. Further, unified memory eliminates a discrete PCIe transfer boundary but introduces shared-memory contention between CPU fallback work, GPU execution, KV cache, and expert-cache activity.

We therefore treat the original paper as design motivation and protocol reference, not as an automatically comparable baseline. A strict replication requires the same checkpoint revision, workload corpus, prompt tokenization, generated-token and stop rules, warmup state, policy configuration, and reported statistic. Those fields have not yet all been recovered for the upstream RTX 4060 row.

3. Native ROCm/HIP Port

The port retains CUDA as a separate runtime path. On ROCm builds, setup detects HIP PyTorch and links the small native extension surface against `libamdhip64` rather than CUDA runtime libraries. A compatibility header maps only the CUDA Runtime API subset already used by FreeToken's pinned-memory and CPU-MoE extensions to HIP. JIT compilation removes NVCC-only flags and uses HIP-compatible launch behavior.

The Python and Triton surfaces require separate treatment. PyTorch exposes ROCm devices through the `torch.cuda` namespace for compatibility, so FreeToken now rejects ROCm before NVIDIA SM capability checks. This prevents `gfx1151` from being misclassified as a hypothetical NVIDIA architecture. CUDA-only optional dependencies and NVIDIA PTX inline assembly are avoided on HIP, with portable Triton implementations used where validated. The GGUF JIT build supplies system ROCm include and library directories only when the PyTorch ROCm wheel omits the necessary developer surface. This supports a native `gfx1151` object without modifying the system ROCm installation.

The resulting server preserves FreeToken's OpenAI-compatible model discovery, streaming, non-streaming, cache, and MoE interfaces. All reported experiments use the ROCm/HIP path. We did not use Vulkan or a CPU-only runner as an implementation substitute.

4. Experimental Methodology

4.1 Platform and runtime

Experiments ran on a GMKtek EVO-X2. Table 1 records the environment observed on 30 August 2026. The port uses ROCm 10, HIP-compiled extensions, and AMD Triton. The Qwen NVFP4 experiment uses the upstream-supported `nvidia/Qwen3.6-35B-A3B-NVFP4` model through native HIP Triton. The same-file Q4 control uses `Qwen3.6-35B-A3B-UD-Q4_K_M.gguf`; Gemma uses `gemma-4-26B_q4_0-it.gguf`.

Table 1. Evaluated-system hardware and software environment. The table reports static platform information. Dynamic measurements such as free memory, temperature, clocks, and active processes are retained per benchmark run in the artifact manifest rather than presented as fixed machine specifications.

Component	Specification
System	GMKtek EVO-X2, SKU `EVO-X2-001`, hardware version 1.0
Firmware	EVO-X2 1.09, 13 September 2025
Processor	AMD Ryzen AI Max+ 395 with Radeon 8060S
CPU topology	16 cores, 32 hardware threads, one NUMA node; boost enabled
CPU frequency	625 MHz minimum and 5.1875 GHz maximum reported by `lscpu`
CPU cache	768 KiB L1d, 512 KiB L1i, 16 MiB L2, and 64 MiB L3
Installed memory	64 GiB LPDDR5, eight 8 GiB Micron devices; 8,532 MT/s rated and 8,000 MT/s configured
Firmware UMA reservation	4 GiB integrated-GPU reservation
Linux-visible host memory	59.46 GiB (`MemTotal`)
ROCm GPU memory pool	56.0 GiB coarse-grained pool reported for `gfx1151`
GPU	AMD Radeon 8060S Graphics, PCI ID `1002:1586`, `gfx1151`
GPU execution resources	40 compute units, wavefront size 32, maximum 32 waves per compute unit
HSA configuration	XNACK disabled; coherent host access reported false
Operating system	Ubuntu 26.04.1 LTS, Linux 7.0.0-30-generic
HIP and compiler	HIP 7.15.26333; AMD Clang 23 from ROCm 10.0
Python runtime	PyTorch `2.13.0+rocm10.0.0`, Triton `3.8.0`, Python 3.12 environment
Storage	Lexar ARES 2 TB NVMe SSD

The 4 GiB firmware reservation is not the FreeToken model-memory budget. It is a preallocated UMA region. The capacity available to a request changes with host activity, runtime overhead, model weights, expert residency, and KV-cache growth. Each scored run therefore records memory pressure and the serving process state separately.

4.2 Metrics and correctness gates

Decode throughput is client-visible streaming throughput: generated completion tokens, excluding the first generated token, divided by the interval from the first to final streamed content token. We keep client-observed TTFT separate from runtime-internal timing. Fixed-length throughput and quality are distinct modes so a system cannot obtain an apparently better rate merely by ending early, emitting hidden reasoning tokens, or silently changing the request.

Every accepted candidate must satisfy all applicable gates: native HIP compilation and execution, successful OpenAI-compatible response, correct tokenizer accounting, deterministic-output or task-quality evidence, and preserved raw artifacts. A microbenchmark gain cannot be accepted if the full model changes the deterministic answer or fails to improve the end-to-end API workload.

4.3 Controls

The Qwen NVFP4 canary uses greedy sampling with a thinking-enabled template and a forced 128-token decode. Its required SHA-1 is `0acef4eab6f4`. The Q4 raw-prompt control sends the same UTF-8 string to both engines' `/v1/completions` endpoint with `temperature=0`, `top_p=1`, `top_k=-1`, streaming enabled, and a 1024-token cap. The prompt SHA-256 is `224f02631165a176e660363fefeb8eb58e5a150271fed72bdc1f90fa39448523`, and both engines report 54 prompt tokens. The expected mathematical result is 70.

5. Results

5.1 Native Qwen NVFP4 serving is functional but does not establish paper parity

The native ROCm/HIP Qwen server passed the deterministic AIME canary with the reference PyTorch router. Three warm quality-matched repeats produced 26.786, 28.422, and 28.431 client-visible tokens/s, for a mean of 27.880 tokens/s. Mean warm TTFT was 409.0 ms for the 54-prompt-token and 127-completion-token request. Every run emitted the required SHA-1.

A ROCm Triton top-k router improved isolated router latency by 1.62 to 1.63x for Qwen's 256-expert top-8 shape, and achieved 29.186 tokens/s in a performance-only NVFP4 workload. However, an end-to-end greedy AIME request changed output hash, so this configuration is rejected for NVFP4 serving. This distinction matters: router-only speed and a transport canary are not a quality-preserving system result.

The 27.880 tokens/s Qwen NVFP4 value is not a direct comparison with the paper's 39.3 tokens/s RTX 4060 result. The underlying model representation is related, but the exact upstream workload and configuration contract is incomplete, and the platforms have materially different memory architecture.

5.2 Matched Q4 Qwen raw-prompt control

Table 2 compares FreeToken and llama.cpp on the same Q4_K_M file, raw prompt, tokenizer count, deterministic sampling, and steady decode rule. Before enabling the in-tree HIP Triton router, FreeToken reached 47.12 tokens/s, 6.3% below the llama.cpp control. With the HIP router enabled, FreeToken reached 50.63 tokens/s while preserving the correct derivation for the expected answer. This is 0.7% above llama.cpp's 50.29 tokens/s.

Engine	Model representation	Prompt tokens	Generated tokens	Steady decode tokens/s	Quality evidence
FreeToken AMD	Qwen Q4_K_M GGUF	54	1023	47.12	Correct derivation for answer 70
FreeToken AMD with native HIP router	Qwen Q4_K_M GGUF	54	1023	50.63	Same correct derivation
llama.cpp ROCm 10	Qwen Q4_K_M GGUF	54	1024	50.29	Same correct derivation

Protocol group	Configuration	Tokens/s	Interpretation
Qwen NVFP4 canary	Reference router	27.88	Three quality-matched runs
Same-file Qwen Q4	FreeToken baseline	47.12	One raw-prompt control
Same-file Qwen Q4	FreeToken plus HIP router	50.63	Correct derivation
Same-file Qwen Q4	llama.cpp ROCm 10	50.29	Correct derivation
Gemma 4 Q4	FreeToken text control	57.05	Fixed arithmetic control

Figure 1. Controlled result overview. The PDF review copy renders this figure with separate visual groups for the Qwen NVFP4 canary, the same-file Qwen Q4 control, and the Gemma text control. The groups must not be read as a single model-format or paper-parity ranking.

This is intentionally a bounded result. Both outputs remained within Qwen's reasoning trace at the 1024-token ceiling, so neither exposed the requested boxed final line. The reasoning nevertheless explicitly derived the two valid bases, whose sum is 70. Future quality experiments should use a larger generation cap or a concise-answer task, plus repeated samples and a task suite.

5.3 Gemma 4 Q4 text control

The native Gemma GGUF path returned `323` for the fixed multiplication prompt, with a prompt hash of `0f65acd07a4f57b2644f7720b725d7795999406b90a9f91486da5effa39bb95d`. The client and local tokenizer agreed on 30 prompt tokens; the response used four completion tokens and reached 57.05 steady decode tokens/s. This validates the text-only loader, canonical template, OpenAI-compatible completion API, and token accounting for this fixed control. It does not alone qualify multimodal handling or long-context behavior.

5.4 Native profiling changes the optimization priority

Profiling the Qwen NVFP4 server under a wheel-compatible ROCm profiler recorded 353,457 dispatches. The trace was intrusive and measured only 15.61 tokens/s, so it is not used for throughput scoring. In its final active window, the largest GPU-time consumer was dense mixed-FP8 `_gemv_splitk_kernel`, not the routed NVFP4 expert kernel. The corresponding measured GPU times were 5,631.844 ms for `_gemv_splitk_kernel`, 1,676.018 ms for `_gemm_kernel`, 1,566.004 ms for `_decode_nvfp4_marlin_kernel`, and 593.192 ms for `fast_index_copy`.

The port also measured its actual Qwen cache-copy path. With one active token, eight routed experts missing, and a 513-slot cache, native HIP copied 13.5 MiB in 0.097 ms, or 146.8 GB/s. The all-hit case took 0.023 ms. Extrapolated across 40 MoE layers, the all-miss copy component is 3.87 ms per decode token, below the approximately 35 ms end-to-end token interval of the accepted NVFP4 configuration. This does not prove copies are irrelevant, but it does rule out treating cache-copy bypass as the first unvalidated optimization.

6. Discussion

The port demonstrates that an edge-native MoE serving design can operate natively on an AMD unified-memory APU. It also shows why portability cannot be reduced to translating CUDA symbols. Correctness is coupled to router selection, tokenizer special-token handling, model representation, cache lifecycle, and the distinction between a microbenchmark and a client-visible request.

The Q4 control is the cleanest current cross-runtime result because it holds the model file and raw prompt constant. It is not a full paper-style agentic comparison, and its 0.7% margin is too small to generalize beyond the stated workload. The NVFP4 path is the closest to FreeToken's original Qwen deployment but has a lower accepted throughput and an unrecovered upstream protocol. It should be described as a native port result, never as an RTX 4060 reproduction or a general AMD performance claim.

Strix Halo also changes FreeToken's systems hypothesis. On a discrete GPU, expert movement crosses PCIe and the CPU and GPU have distinct primary memory systems. On this APU, CPU fallback, GPU kernels, expert residency, and KV growth compete for a shared memory pool. The next policy should therefore be based on measured contention curves over cache size, KV allocation, and CPU contribution. It should not assume that a PCIe-oriented hybrid rule or pinned-buffer strategy transfers unchanged to UMA.

7. Artifact Availability and Independent Extension

The AMD ROCm/HIP port is developed under Apache-2.0 at `https://github.com/dbourdea/FreeToken`, branch `amd-rocm-gfx1151`. This release candidate is based on public branch tip `a937862f171900bd5d1d207c8ff59b40a15ce742`, verified on 30 August 2026. The white-paper package and portable reproduction tools are not yet committed to that branch, and no immutable tag or DOI exists. Before publication, commit the complete package, archive an immutable tag, and replace this statement with the tag and version DOI. The release candidate includes HIP portability tests, Qwen and Gemma controls, and the read-only collector `scripts/reproduce/collect_host_manifest.sh`, which redacts the hostname by default and does not change service or host configuration.

To make the work independently reproducible, the archival release must contain four separable components. First, the source archive must include the pinned commit, an environment lockfile, and a machine-readable schema for the run manifest. Second, the workload archive must contain the exact prompt text, request settings, tokenizer expectations, scoring code, and result schema. Third, the result archive must contain raw streaming timestamps, response text or output hashes as appropriate, telemetry, logs sanitized for credentials and host identifiers, and the script that generates each manuscript table. Fourth, model provenance must specify publisher, revision, byte count, SHA-256, and license, while directing users to obtain weights from the original publisher rather than redistributing weights without permission.

The original host-specific helpers preserve a protected local service and use host-specific model paths. They remain appropriate for evidence capture on the test system, but they are not the public entry point. The public artifact now provides a parameterized host collector and a loopback-only API client at `benchmarks/reproduce/run_local_api_benchmark.py`. The client accepts explicit model, tokenizer, prompt,

visible-text quality gate, sample count, and artifact path; it cannot send traffic to a LAN or public address and never starts or stops a server. Future release work must add parameterized model-launch recipes, but public runners must continue to avoid user-specific home paths, LAN addresses, or an assumed production service.

Independent contributors can extend this artifact in four well-defined directions: add a host manifest and clean benchmark matrix for another AMD target; implement a unified-memory-aware expert-cache policy; contribute a profile-ranked HIP kernel candidate; or expand the deterministic and task-level quality suite. Every extension should retain raw evidence, preserve the stated output gate, and report rejected as well as accepted candidates.

8. Limitations and Reproducibility

This study reports a single `gfx1151` host, a small number of controlled workloads, and no 24-hour endurance result. It does not establish broad AMD support, cross-device generalization, agentic quality equivalence, or strict parity with the upstream paper. Some currently useful comparisons still involve different representations, such as NVFP4 versus Q4_K_M, and must not be interpreted as architecture-independent engine rankings.

Future work should recover the upstream Qwen benchmark contract, complete a five-sample paper-matched matrix, add long-context and multi-turn quality controls, measure UMA contention directly, and repeat any accepted optimization on a second clean-host matrix. The public artifact protocol in Section 7 is the required path for reproducing and extending those experiments.

9. Conclusion

We ported FreeToken to native ROCm/HIP execution on AMD Strix Halo and evaluated it with a claim discipline suited to an evolving edge-serving system. The port compiles and serves through HIP, preserves CUDA as a separate path, and passes deterministic Qwen and Gemma controls. In a same-file Q4 Qwen control, the native HIP router produced 50.63 tokens/s versus 50.29 tokens/s for llama.cpp ROCm 10, while a faster but output-changing NVFP4 router was rejected. Profiling identifies dense FP8 decode as the leading current target and shows that unified-memory expert-cache copies are not, by themselves, the dominant observed decode cost. The result is a reproducible foundation for further UMA-aware optimization, not a claim of paper replication or universal AMD superiority.

References

[1] Shuo Yang et al. *FreeToken: Efficient Edge-Native MoE Serving with Bandwidth-Adaptive Execution.* arXiv:2608.16157, 2026.

[2] Georgi Gerganov et al. *llama.cpp.* https://github.com/ggml-org/llama.cpp.

[3] AMD. *ROCm Documentation.* https://rocm.docs.amd.com/.

[4] FreeToken AMD contributors. *FreeToken AMD ROCm/HIP Port for Strix Halo: Technical White Paper and Artifact Release Candidate v0.1.0-rc1.* Branch `amd-rocm-gfx1151`, commit `a937862f171900bd5d1d207c8ff59b40a15ce742`; tag and DOI pending, 2026.

[5] Apache Software Foundation. *Apache License, Version 2.0.* https://www.apache.org/licenses/LICENSE-2.0.

Appendix A. Claim ledger for reviewers

Claim	Evidence status	Boundary
Native AMD execution	Established	HIP-compiled extension and native ROCm/HIP server, no Vulkan or CPU substitute
Qwen NVFP4 deterministic serving	Established for canary	Reference-router AIME hash, not a complete task-quality suite
Qwen NVFP4 27.88 tokens/s	Measured	Three warm quality-matched runs, 54-prompt-token/127-completion-token canary
Qwen Q4 50.63 tokens/s	Measured	One same-file raw-prompt control with native HIP router
Faster than llama.cpp for Qwen Q4 control	Bounded	0.7% on one stated steady-decode control, not a general ranking
Gemma 4 Q4 57.05 tokens/s	Measured	Fixed text arithmetic control
Reproduces FreeToken 39.3 tokens/s RTX 4060 result	Not established	Upstream workload and configuration contract incomplete
General Strix Halo or AMD advantage	Not established	One host and limited workload matrix